31

信号量

我们现在知道，要解决各种重要而有趣的并发问题，锁（lock）和条件变量（condition variable）缺一不可。多年前，最早意识到这一点的人之一就是 **Edsger Dijkstra**（不过确切的历史很难考证 [GR92]）。他为人所知的成就有很多：图论中著名的“最短路径”算法 [D59]；早年一篇讨论结构化编程的论战文章《Goto Statements Considered Harmful》[D68a]（这标题起得真好！）；还有本章的主角：他提出的一种叫作**信号量**（semaphore）的同步原语 [D68b, D72]。事实上，Dijkstra 和同事发明信号量，是想用一种原语处理所有与同步有关的事情；你会看到，信号量既可以当锁用，也可以当条件变量用。

关键问题：如何使用信号量
如何用信号量代替锁和条件变量？信号量的定义是什么？什么是二值信号量？用锁和条件变量构建信号量容易吗？反过来，用信号量构建锁和条件变量呢？

31.1 信号量的定义

信号量是一个带有整数值的对象，我们可以用两个例程来操作它；在 POSIX 标准中，这两个例程是 `sem_wait()` 和 `sem_post()`[1]。信号量的初始值决定了它的行为，所以在调用任何其他例程操作信号量之前，必须先把它初始化为某个值，如图 31.1 中的代码所示。

在图中，我们声明了一个信号量 `s`，给第三个参数传入 1，把它初始化为 1。在后面的所有例子中，`sem_init()` 的第二个参数都设为 0，表示这个信号量由同一进程内的各个线

```
1 #include <semaphore.h>
2 sem_t s;
3 sem_init(&s, 0, 1);
```

图 31.1：**初始化信号量**

[1]历史上，Dijkstra 把 `sem_wait()` 叫作 `P()`，把 `sem_post()` 叫作 `V()`。这两个简称来自荷兰语单词；有意思的是，它们究竟源自哪两个荷兰语单词，说法前后有过变化。最初，`P()` 来自“passering”（通过），`V()` 来自“vrijgave”（释放）；后来 Dijkstra 又写道，`P()` 来自“prolaag”，它是“probeer”（荷兰语的“尝试”）和“verlaag”（“减少”）的合成词，而 `V()` 来自“verhoog”，意思是“增加”。有时人们也把它们叫作 down 和 up。拿荷兰语的说法去让朋友刮目相看吧，或者把他们弄糊涂，或者两者兼得。详见 https://news.ycombinator.com/item?id=8761539。

```
1 int sem_wait(sem_t *s) {
2     把信号量 s 的值减 1
3     如果信号量 s 的值为负，就等待
4 }
5
6 int sem_post(sem_t *s) {
7     把信号量 s 的值加 1
8     如果有一个或多个线程在等待，就唤醒其中一个
9 }
```

图 31.2：**信号量：wait 和 post 的定义**

程共享。信号量也可以用来同步不同进程对资源的访问，这时第二个参数要传别的值，详见手册页。

信号量初始化之后，我们可以调用 `sem_wait()` 或 `sem_post()` 这两个函数来操作它。这两个函数的行为见图 31.2。

我们暂时不关心这两个例程如何实现。实现它们显然要格外小心：多个线程会同时调用 `sem_wait()` 和 `sem_post()`，这些临界区（critical section）自然需要妥善管理。现在我们先关注怎样使用这些原语，以后也许再讨论它们是怎么构建的。

我们来说说这两个接口的几个要点。第一，`sem_wait()` 要么立即返回（因为调用 `sem_wait()` 时信号量的值大于或等于 1），要么让调用者暂停执行，等待之后的 post 操作。当然，可能有多个线程调用 `sem_wait()`，它们会全部排队，等着被唤醒。

第二，`sem_post()` 不像 `sem_wait()` 那样要等待某个条件成立。它只是把信号量的值加 1，然后，如果有线程在等待唤醒，就唤醒其中一个。

第三，信号量的值为负时，其绝对值等于正在等待的线程数 [D68b]。信号量的使用者一般看不到这个值，但这个不变式（invariant）还是值得了解的，也许能帮你记住信号量是怎样工作的。

信号量内部看起来可能存在竞态条件（race condition，也叫竞争条件），这一点（暂时）不必担心，先假设信号量的操作都是原子的。我们很快就会用锁和条件变量做到这一点。

31.2 二值信号量（锁）

现在可以使用信号量了。第一个用法是我们已经熟悉的：把信号量当锁用。代码片段见图 31.3：我们只是用一对 `sem_wait()`/`sem_post()` 把要保护的临界区包起来。不过，要让它正确工作，关键在于信号量 `m` 的初始值（图中初始化为 `X`）。`X` 应该是多少？

```
1 sem_t m;
2 sem_init(&m, 0, X); // 初始化为 X；X 应该是多少？
3
4 sem_wait(&m);
5 // 这里是临界区
6 sem_post(&m);
```

图 31.3：**二值信号量（也就是锁）**

信号量的值	线程 0	线程 1
1		
1	调用 sem_wait()	
0	sem_wait() 返回	
0	（临界区）	
0	调用 sem_post()	
1	sem_post() 返回	

图 31.4：**线程轨迹：单个线程使用信号量**

……（继续往下读之前，先自己想一想）……

回头看看前面 sem_wait() 和 sem_post() 的定义，就能看出初始值应该是 1。

为了说清楚，我们设想一个有两个线程的场景。第一个线程（线程 0）调用 sem_wait()，它先把信号量的值减 1，变成 0。然后，只有当值不大于或等于 0 时，它才会等待。因为值是 0，sem_wait() 直接返回，调用它的线程继续运行；线程 0 现在可以进入临界区了。如果线程 0 在临界区里时没有其他线程试图获取锁，那么它调用 sem_post() 时，只是把信号量的值恢复为 1（也不会唤醒等待的线程，因为根本没有）。图 31.4 展示了这个场景的轨迹。

更有意思的情况是：线程 0 “持有锁”（即已经调用了 sem_wait()，但还没调用 sem_post()）时，另一个线程（线程 1）调用 sem_wait()，试图进入临界区。这时，线程 1 会把信号量的值减到 –1，于是开始等待（让自己睡眠，让出处理器）。线程 0 再次运行后，最终会调用 sem_post()，把信号量的值加回 0，然后唤醒等待的线程（线程 1），线程 1 随后就能获取锁了。线程 1 结束时，会再次把信号量的值加 1，让它恢复为 1。

图 31.5 展示了这个例子的轨迹。除了线程的动作，图中还标出了每个线程的**调度状态**（scheduler state）：运行（线程正在运行）、就绪（可以运行，但没有在运行）和睡眠（线程被阻塞）。注意，线程 1 试图获取已被持有的锁时，就进入了睡眠状态；只有等线程 0 再次运行，线程 1 才能被唤醒，才有可能再次运行。

如果你想自己推演一个例子，可以试试多个线程排队等锁的场景。在这样的轨迹中，信号量的值会是多少？

这样，我们就能把信号量当锁用了。因为锁只有两种状态（被持有和未被持有），所以有时把当作锁用的信号量叫作二**值信号量**（binary semaphore）。注意，如果你只按这种二值的方式使用信号量，它的实现可以比我们这里介绍的通用信号量更简单。

31.3 用信号量控制顺序

在并发程序中，信号量也可以用来安排事件发生的先后顺序。例如，一个线程可能想等某个链表变为非空，再从中删除一个元素。在这种用法中，常见的情形是：一个线程等待某件事发生，另一个线程让这件事发生，然后发信号告知它已经发生，从而唤醒等待的线程。于是，我们把信号量当作一种**顺序**（ordering）原语来用（和前面使用**条件变量**的方式类似）。

值	线程 0	状态	线程 1	状态
1		运行		就绪
1	调用 sem_wait()	运行		就绪
0	sem_wait() 返回	运行		就绪
0	(临界区开始)	运行		就绪
0	中断；切换→*T1*	就绪		运行
0		就绪	调用 sem_wait()	运行
−1		就绪	sem 减 1	运行
−1		就绪	(sem<0)→睡眠	睡眠
−1		运行	切换→*T0*	睡眠
−1	(临界区结束)	运行		睡眠
−1	调用 sem_post()	运行		睡眠
0	sem 加 1	运行		睡眠
0	wake(T1)	运行		就绪
0	sem_post() 返回	运行		就绪
0	中断；切换→*T1*	就绪		运行
0		就绪	sem_wait() 返回	运行
0		就绪	(临界区)	运行
0		就绪	调用 sem_post()	运行
1		就绪	sem_post() 返回	运行

图 31.5：**线程轨迹：两个线程使用信号量**

下面是一个简单的例子。设想一个线程创建了另一个线程，然后想等它执行完（图 31.6）。程序运行时，我们希望看到如下输出：

```
1  sem_t s;
2
3  void *child(void *arg) {
4      printf("child\n");
5      sem_post(&s); // 在这里发信号：子线程执行完了
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     sem_init(&s, 0, X); // X 应该是多少?
11     printf("parent: begin\n");
12     pthread_t c;
13     Pthread_create(&c, NULL, child, NULL);
14     sem_wait(&s); // 在这里等待子线程
15     printf("parent: end\n");
16     return 0;
17 }
```

图 31.6：**父线程等待子线程**

值	父线程	状态	子线程	状态
0	create(Child)	运行	（子线程已存在，可以运行）	就绪
0	调用 sem_wait()	运行		就绪
−1	sem 减 1	运行		就绪
−1	(sem<0)→睡眠	睡眠		就绪
−1	切换→子线程	睡眠	子线程运行	运行
−1		睡眠	调用 sem_post()	运行
0		睡眠	sem 加 1	运行
0		就绪	wake(Parent)	运行
0		就绪	sem_post() 返回	运行
0		就绪	中断→父线程	就绪
0	sem_wait() 返回	运行		就绪

图 31.7：**线程轨迹：父线程等待子线程（情况 1）**

```
parent: begin
child
parent: end
```

问题是，怎样用信号量达到这个效果。其实答案相对容易理解。从代码中可以看到，父线程只需调用 sem_wait()，子线程调用 sem_post()，以此等待“子线程执行完毕”这个条件成立。但问题又来了：这个信号量的初始值应该是多少？

（同样，先在这里想一想，别急着往下读）

答案当然是：信号量的值应该设为 0。需要考虑两种情况。第一种，假设父线程创建了子线程，但子线程还没有运行（也就是说，它在就绪队列里，但没有在运行）。这时（图 31.7，第 5 页），父线程会在子线程调用 sem_post() 之前调用 sem_wait()；我们希望父线程等待子线程运行。要做到这一点，唯一的办法是让信号量的值不大于 0，所以初始值是 0。父线程运行，把信号量减为 −1，然后等待（睡眠）。子线程最终运行时，会调用 sem_post()，把信号量的值加到 0，并唤醒父线程；父线程随后从 sem_wait() 返回，程序结束。

值	父线程	状态	子线程	状态
0	create(Child)	运行	（子线程已存在，可以运行）	就绪
0	中断→子线程	就绪	子线程运行	运行
0		就绪	调用 sem_post()	运行
1		就绪	sem 加 1	运行
1		就绪	wake(nobody)	运行
1		就绪	sem_post() 返回	运行
1	父线程运行	运行	中断→父线程	就绪
1	调用 sem_wait()	运行		就绪
0	sem 减 1	运行		就绪
0	(sem≥0)→不睡眠	运行		就绪
0	sem_wait() 返回	运行		就绪

图 31.8：**线程轨迹：父线程等待子线程（情况 2）**

提示：设置信号量的值
我们已经看到了两个初始化信号量的例子。第一个例子把值设为 1，把信号量当锁用；第二个例子设为 0，用信号量控制顺序。那么，信号量的初始化有什么通用规则吗？
有一种简单的思路（这要感谢 Perry Kivolowitz）：想一想初始化之后，你愿意马上分出去多少份资源。对锁来说是 1，因为初始化之后，你愿意让锁马上被锁上（分出去）。对控制顺序的情况来说是 0，因为一开始没有东西可以分出去；只有等子线程执行完，资源才被创造出来，这时值才加到 1。以后遇到信号量问题时，试试这种思路，看看有没有帮助。

第二种情况（图 31.8）是，子线程在父线程有机会调用 `sem_wait()` 之前就运行完了。这时，子线程先调用 `sem_post()`，把信号量的值从 0 加到 1。等父线程有机会运行时，它调用 `sem_wait()`，发现信号量的值是 1；于是父线程把值减为 0，不用等待就从 `sem_wait()` 返回，同样达到了预期的效果。

31.4 生产者/消费者（有界缓冲区）问题

本章要解决的下一个问题叫作**生产者/消费者**（producer/consumer）问题，有时也叫**有界缓冲区**（bounded buffer）问题 [D72]。上一章讲条件变量时已经详细描述过这个问题，细节请看那一章。

第一次尝试

第一次尝试解决这个问题时，我们引入了两个信号量 `empty` 和 `full`，线程用它们分别表示缓冲区的某一项已被清空或已被填满。`put` 和 `get` 例程的代码见图 31.9，我们为生产者/消费者问题写的第一版解法见图 31.10（第 7 页）。

```
1  int buffer[MAX];
2  int fill = 0;
3  int use  = 0;
4
5  void put(int value) {
6      buffer[fill] = value;     // F1 行
7      fill = (fill + 1) % MAX; // F2 行
8  }
9
10 int get() {
11     int tmp = buffer[use];    // G1 行
12     use = (use + 1) % MAX;    // G2 行
13     return tmp;
14 }
```

图 31.9：**put 和 get 例程**

 © 2008–25, ARPACI-DUSSEAU

```
1  sem_t empty;
2  sem_t full;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          sem_wait(&empty);          // P1 行
8          put(i);                    // P2 行
9          sem_post(&full);           // P3 行
10     }
11 }
12
13 void *consumer(void *arg) {
14     int tmp = 0;
15     while (tmp != -1) {
16         sem_wait(&full);           // C1 行
17         tmp = get();               // C2 行
18         sem_post(&empty);          // C3 行
19         printf("%d\n", tmp);
20     }
21 }
22
23 int main(int argc, char *argv[]) {
24     // ...
25     sem_init(&empty, 0, MAX); // MAX 个缓冲区为空
26     sem_init(&full, 0, 0);    // 0 个缓冲区是满的
27     // ...
28 }
```

图 31.10：**加上 full 和 empty 条件**

在这个例子中，生产者先等待某个缓冲区变空，以便往里面放数据；消费者也类似，先等待某个缓冲区被填满，再使用它。我们先设想 `MAX=1`（数组里只有一个缓冲区），看看这样行不行。

再设想有两个线程，一个生产者，一个消费者。我们来考察单个 CPU 上的一个具体场景。假设消费者先运行。于是，消费者执行到图 31.10 中的 C1 行，调用 `sem_wait(&full)`。因为 `full` 初始化为 0，这次调用会把 `full` 减为 –1，阻塞消费者，等待另一个线程对 `full` 调用 `sem_post()`，这正是我们想要的。

然后假设生产者运行。它执行到 P1 行，调用 `sem_wait(&empty)` 例程。和消费者不同，生产者会顺利通过这一行，因为 `empty` 初始化为 `MAX`（这里是 1）。于是 `empty` 减为 0，生产者把一个数据放进缓冲区的第一项（P2 行）。接着生产者执行到 P3 行，调用 `sem_post(&full)`，把 `full` 信号量的值从 –1 改为 0，并唤醒消费者（例如把它从阻塞状态移到就绪状态）。

接下来可能发生两种情况之一。如果生产者继续运行，它会绕回循环开头，再次执行到 P1 行。不过这一次它会阻塞，因为 `empty` 信号量的值是 0。如果生产者被中断，消费者开

```
1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&mutex);          // P0 行（新增）
5          sem_wait(&empty);          // P1 行
6          put(i);                    // P2 行
7          sem_post(&full);           // P3 行
8          sem_post(&mutex);          // P4 行（新增）
9      }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&mutex);          // C0 行（新增）
16         sem_wait(&full);           // C1 行
17         int tmp = get();           // C2 行
18         sem_post(&empty);          // C3 行
19         sem_post(&mutex);          // C4 行（新增）
20         printf("%d\n", tmp);
21     }
22 }
```

图 31.11：**加上互斥（错误的做法）**

始运行，消费者就会从 `sem_wait(&full)`（C1 行）返回，发现缓冲区是满的，然后取走数据。无论哪种情况，都达到了预期的效果。

你可以用更多线程（例如多个生产者和多个消费者）试试这个例子，它应该仍然能正常工作。

现在设想 `MAX` 大于 1（比如 `MAX=10`），并且有多个生产者和多个消费者。这下就有问题了：出现了竞态条件。你看出它在哪里了吗？（花点时间找一找。）如果没看出来，给个提示：仔细看看 `put()` 和 `get()` 的代码。

好，我们来弄清楚这个问题。设想两个生产者（Pa 和 Pb）几乎同时调用 `put()`。假设生产者 Pa 先运行，刚开始填充缓冲区的第一项（F1 行，`fill=0`）。Pa 还没来得及把 `fill` 计数器加到 1，就被中断了。生产者 Pb 开始运行，它在 F1 行也把数据放进了 `buffer` 的第 0 个元素，这意味着原来的数据被覆盖了！这可不行，我们不希望丢失生产者的任何数据。

解决办法：加上互斥

可以看到，我们这里忘了*互斥*（mutual exclusion）。填充缓冲区和递增缓冲区索引这两步构成一个临界区，必须小心保护起来。所以，我们请出老朋友二值信号量，加上一些锁。图 31.11 是我们的尝试。

现在，我们在整个 `put()/get()` 部分的代码周围加了锁（见注释中标着“新增”的行）。这个思路看起来是对的，但还是不行。为什么？死锁（deadlock）。为什么会发生死锁？花点时间想一想，试着找出一种会出现死锁的情况。要让程序死锁，必须依次发生哪些步骤？

避免死锁

好了，你已经想出来了，下面揭晓答案。设想有两个线程，一个生产者，一个消费者。消费者先运行。它获取互斥锁（C0 行），然后对 `full` 信号量调用 `sem_wait()`（C1 行）；因为还没有数据，这次调用让消费者阻塞，于是让出 CPU。但要注意，消费者仍然持有锁。

接着生产者运行。它有数据要生产，只要它能继续运行，就能唤醒消费者线程，一切都会顺利。遗憾的是，它做的第一件事就是对二值的 `mutex` 信号量调用 `sem_wait()`（P0 行），而锁已经被持有了。于是，生产者现在也卡在那里等待。

这里有一个简单的循环。消费者持有互斥锁，并在等待有人对 `full` 发信号；生产者本可以对 `full` 发信号，却在等待互斥锁。于是，生产者和消费者互相等待，都卡住了：这就是典型的死锁。

终于有了可行的方案

要解决这个问题，只需缩小锁的作用范围。图 31.12（第 9 页）给出了正确的方案。可以看到，我们只是把互斥锁的获取和释放移到紧挨着临界区的地方，对 `full` 和 `empty` 的等待和发信号代码留在锁的外面[2]。这样就得到了一个简单而可用的有界缓冲区，这是多线程程序中很常用的模式。现在就弄懂它，以后用得上。未来很多年你都会感谢我们。至少，当期末考试或者面试中出现同样的问题时，你会感谢我们的。

```
1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&empty);          // P1 行
5          sem_wait(&mutex);          // P1.5 行（加锁）
6          put(i);                    // P2 行
7          sem_post(&mutex);          // P2.5 行（解锁）
8          sem_post(&full);           // P3 行
9      }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&full);           // C1 行
16         sem_wait(&mutex);          // C1.5 行（加锁）
17         int tmp = get();           // C2 行
18         sem_post(&mutex);          // C2.5 行（解锁）
19         sem_post(&empty);          // C3 行
20         printf("%d\n", tmp);
21     }
22 }
```

图 31.12：**加上互斥（正确的做法）**

[2]其实，从模块化的角度考虑，把互斥锁的获取和释放放在 `put()` 和 `get()` 函数内部可能更自然。

31.5 读写锁

另一个经典问题源于这样一种需求：我们希望有一种更灵活的加锁原语，能针对不同类型的数据结构访问使用不同类型的锁。例如，设想一个链表上有许多并发操作，包括插入和简单的查找。插入会改变链表的状态（因此用传统的临界区是合理的），而查找只是读取数据结构；只要能保证没有正在进行的插入，就可以让多个查找同时进行。我们接下来要开发的这种特殊的锁就是为了支持这类操作，它叫作**读写锁**（reader-writer lock）[CHP71]。这种锁的代码见图 31.13（第 11 页）。

代码相当简单。如果某个线程想更新这个数据结构，它应该调用一对新的同步操作：用 `rwlock_acquire_writelock()` 获取写锁，用 `rwlock_release_writelock()` 释放写锁。这两个函数内部只是用 `writelock` 信号量来确保只有一个写者能获取锁，从而进入临界区更新数据结构。

更有意思的是获取和释放读锁的那对例程。获取读锁时，读者先获取 `lock`，然后递增 `readers` 变量，以记录当前有多少读者在访问数据结构。`rwlock_acquire_readlock()` 中接下来的关键一步发生在第一个读者获取锁的时候：这时，这个读者还要对 `writelock` 信号量调用 `sem_wait()`，获取写锁，然后再调用 `sem_post()` 释放 `lock`。

因此，一旦有读者获取了读锁，其他读者也可以获取读锁；但是，任何想获取写锁的线程都必须等到所有读者都结束；最后一个退出临界区的读者会对“writelock”调用 `sem_post()`，从而让等待中的写者获取锁。

这种方法能（如愿）工作，但也有一些缺点，尤其是在公平性（fairness）方面。具体来说，读者很容易让写者饿死。这个问题有更精巧的解决方案；也许你能想出一种更好的实现？提示：想想一旦有写者在等待，怎样阻止更多读者进入。

最后要指出，读写锁要谨慎使用。它们往往带来更多开销（实现越复杂越是如此），所以和直接使用简单快速的加锁原语相比，最终并不能提升性能 [CB08]。无论如何，它们再次展示了怎样以有趣而有用的方式使用信号量。

31.6 哲学家就餐问题

Dijkstra 提出并解决的并发问题中，最著名的一个叫作**哲学家就餐问题**（dining philosopher's problem）[D71]。这个问题之所以出名，是因为它好玩，在智力上也有些意思，不过实用价值不高。可它实在太有名了，我们不得不把它放进来；事实上，你可能在某次面试中被问到它，要是答不上来、没拿到工作，你一定会恨死你的操作系统教授。反过来，如果你拿到了工作，欢迎给你的操作系统教授寄一封感谢信，或者一些股票期权。

这个问题的基本设定是这样的（见图 31.14）：假设有五位“哲学家”围坐在一张桌子旁。每两位哲学家之间放着一把叉子（因此一共五把）。哲学家们有时在思考，这时不需要叉子；有时在吃饭。哲学家要吃饭，需要两把叉子，也就是自己左边和右边的各一把。正是对这些叉子的争用，以及由此产生的同步问题，让它成了我们在并发编程中研究的问题。

下面是每位哲学家的基本循环，假设每位哲学家都有一个唯一的线程标识符 `p`，取值为 0 到 4（含 4）：

```
1  typedef struct _rwlock_t {
2      sem_t lock;      // 二值信号量（基本的锁）
3      sem_t writelock; // 允许一个写者/多个读者
4      int   readers;   // 临界区内的读者数
5  } rwlock_t;
6
7  void rwlock_init(rwlock_t *rw) {
8      rw->readers = 0;
9      sem_init(&rw->lock, 0, 1);
10     sem_init(&rw->writelock, 0, 1);
11 }
12
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14     sem_wait(&rw->lock);
15     rw->readers++;
16     if (rw->readers == 1) // 第一个读者获取写锁
17         sem_wait(&rw->writelock);
18     sem_post(&rw->lock);
19 }
20
21 void rwlock_release_readlock(rwlock_t *rw) {
22     sem_wait(&rw->lock);
23     rw->readers--;
24     if (rw->readers == 0) // 最后一个读者释放写锁
25         sem_post(&rw->writelock);
26     sem_post(&rw->lock);
27 }
28
29 void rwlock_acquire_writelock(rwlock_t *rw) {
30     sem_wait(&rw->writelock);
31 }
32
33 void rwlock_release_writelock(rwlock_t *rw) {
34     sem_post(&rw->writelock);
35 }
```

图 31.13：**一个简单的读写锁**

```
while (1) {
    think();
    get_forks(p);
    eat();
    put_forks(p);
}
```

关键的挑战在于，怎样编写 `get_forks()` 和 `put_forks()` 例程，使得不会发生死锁，没有哪位哲学家会饿死、永远吃不上饭，并且并发度高（也就是让尽可能多的哲学家同时吃饭）。

参照 Downey 的解法 [D08]，我们会用几个辅助函数来帮助求解。它们是：

提示：简单笨办法可能更好（Hill 定律）

永远不要低估这样一个观点：简单笨拙的办法可能就是最好的办法。就加锁而言，有时简单的自旋锁（spin lock）效果最好，因为它容易实现，速度又快。读写锁之类的东西听起来很酷，但它们很复杂，而复杂可能就意味着慢。所以，总是先试试简单笨拙的办法。这种崇尚简单的思想在很多地方都能看到。较早的一个出处是 Mark Hill 的博士论文 [H87]，它研究的是如何为 CPU 设计缓存。Hill 发现，简单的直接映射缓存（direct-mapped cache）比花哨的组相联（set-associative）设计效果更好（原因之一是，在缓存中，设计越简单，查找就越快）。Hill 把他的工作简洁地概括为："大而笨更好。"因此，我们把这条类似的建议叫作 **Hill 定律**（Hill's Law）。

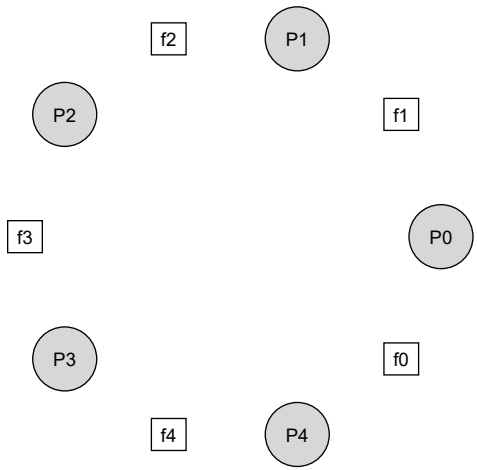


图 31.14：**哲学家就餐问题**

```
int left(int p)  { return p; }
int right(int p) { return (p + 1) % 5; }
```

哲学家 p 想引用左边的叉子时，只需调用 `left(p)`。类似地，调用 `right(p)` 就能引用哲学家 p 右边的叉子；其中的取模运算处理了一种特殊情况：最后一位哲学家（`p=4`）要拿右边的叉子，也就是叉子 0。

我们还需要一些信号量来解决这个问题。假设有五个信号量，每把叉子一个：`sem_t forks[5]`。

有问题的方案

我们来尝试第一个解法。假设把（`forks` 数组中的）每个信号量都初始化为 1，并且每位哲学家都知道自己的编号（`p`）。于是可以写出 `get_forks()` 和 `put_forks()` 例程（图 31.15，第 13 页）。

```
1 void get_forks(int p) {
2     sem_wait(&forks[left(p)]);
3     sem_wait(&forks[right(p)]);
4 }
5
6 void put_forks(int p) {
7     sem_post(&forks[left(p)]);
8     sem_post(&forks[right(p)]);
9 }
```

图 31.15：**get_forks() 和 put_forks() 例程**

这个（有问题的）方案背后的直觉是这样的：要拿叉子，只需对每把叉子各加一把“锁”：先拿左边的，再拿右边的。吃完以后，再把它们放下。很简单，对吧？可惜，在这里，简单就意味着行不通。你能看出会出现什么问题吗？想一想。

问题在于**死锁**。如果在任何一位哲学家拿到右边的叉子之前，每位哲学家都恰好先拿起了左边的叉子，那么每个人都会拿着一把叉子，永远等着另一把。具体来说，哲学家 0 拿起叉子 0，哲学家 1 拿起叉子 1，哲学家 2 拿起叉子 2，哲学家 3 拿起叉子 3，哲学家 4 拿起叉子 4；所有叉子都被拿走了，所有哲学家都卡在那里，等着另一位哲学家手里的叉子。我们很快会更详细地研究死锁；现在可以肯定地说，这不是一个可行的方案。

一种解决办法：打破依赖

解决这个问题最简单的办法，是改变至少一位哲学家拿叉子的方式；事实上，Dijkstra 本人就是这样解决的。具体来说，假设哲学家 4（编号最大的那位）拿叉子的顺序和其他人不同（图 31.16）；`put_forks()` 的代码保持不变。

因为最后一位哲学家先拿右边、再拿左边，所以不会出现每位哲学家都拿着一把叉子、卡着等另一把的情况；等待的循环被打破了。仔细想想这个方案会带来什么结果，说服自己它确实可行。

类似的“著名”问题还有一些，比如**吸烟者问题**（cigarette smoker's problem）和**睡眠理发师问题**（sleeping barber problem）。它们大多只是让人思考并发的由头，有些名字起得很有意思。如果你想了解更多，或者只是想多练习用并发的方式思考，可以去查一查 [D08]。

```
1 void get_forks(int p) {
2     if (p == 4) {
3         sem_wait(&forks[right(p)]);
4         sem_wait(&forks[left(p)]);
5     } else {
6         sem_wait(&forks[left(p)]);
7         sem_wait(&forks[right(p)]);
8     }
9 }
```

图 31.16：**打破 get_forks() 中的依赖**

提示：泛化要小心

在系统设计中，泛化这种抽象技术可能相当有用：一个好的想法稍微扩展一下，就能解决更大一类问题。不过，泛化时要小心；正如 Lampson 告诫我们的：“不要泛化；泛化的东西一般都是错的。”[L83]

可以把信号量看作锁和条件变量的泛化；但是，真的需要这样的泛化吗？而且，考虑到在信号量之上实现条件变量的难度，也许这种泛化并不像你想的那样“泛”。

31.7 线程节流

信号量还有另一个简单的用法，偶尔会用到，所以这里也介绍一下。具体问题是：程序员怎样防止“太多”线程同时做某件事，把系统拖垮？答案是：为“太多”定一个阈值，然后用信号量限制同时执行那段代码的线程数。我们把这种方法叫作**节流**（throttling）[T99]，并把它看作**准入控制**（admission control）的一种形式。

我们来看一个更具体的例子。设想你创建了几百个线程，并行地处理某个问题。然而，在代码的某个部分，每个线程都要申请大量内存来完成一部分计算；我们把这部分代码叫作*内存密集区*（memory-intensive region）。如果所有线程同时进入内存密集区，所有内存分配请求加起来就会超过机器的物理内存总量。结果，机器开始抖动（thrashing，也就是在内存和磁盘之间来回换页），整个计算就会慢得像蜗牛爬。

一个简单的信号量就能解决这个问题。把信号量的值初始化为你希望同时进入内存密集区的最大线程数，然后在这个区域前后放上 `sem_wait()` 和 `sem_post()`，信号量就能自然地限制同时处在这段危险代码中的线程数。

31.8 如何实现信号量

最后，我们用底层的同步原语，也就是锁和条件变量，来构建我们自己的信号量，它叫作……（*此处应有鼓声*）……**Zemaphore**。这项任务相当简单，见图 31.17（第 15 页）。

在这段代码中，我们只用了一个锁和一个条件变量，外加一个记录信号量值的状态变量。自己研究这段代码，直到真正弄懂为止。动手吧！

我们的 Zemaphore 和 Dijkstra 定义的纯粹的信号量有一个细微的区别：我们没有维持“信号量的值为负时反映等待线程数”这个不变式；实际上，这个值永远不会小于零。这种行为更容易实现，也和目前 Linux 的实现一致。

有意思的是，用信号量构建条件变量要棘手得多。一些经验丰富的并发程序员曾在 Windows 环境中尝试这样做，结果出了各种各样的 bug [B04]。你自己试试，看能不能弄明白，为什么用信号量构建条件变量比看上去更有挑战性。

31.9 小结

信号量是编写并发程序的一种强大而灵活的原语。由于它简单实用，有些程序员只用信号量，完全不用锁和条件变量。

```
typedef struct __Zem_t {
    int value;
    pthread_cond_t cond;
    pthread_mutex_t lock;
} Zem_t;

// 只能由一个线程调用
void Zem_init(Zem_t *s, int value) {
    s->value = value;
    Cond_init(&s->cond);
    Mutex_init(&s->lock);
}

void Zem_wait(Zem_t *s) {
    Mutex_lock(&s->lock);
    while (s->value <= 0)
        Cond_wait(&s->cond, &s->lock);
    s->value--;
    Mutex_unlock(&s->lock);
}

void Zem_post(Zem_t *s) {
    Mutex_lock(&s->lock);
    s->value++;
    Cond_signal(&s->cond);
    Mutex_unlock(&s->lock);
}
```

图 31.17：**用锁和条件变量实现 Zemaphore**

本章只介绍了几个经典问题及其解法。如果你有兴趣了解更多，可以参考很多其他资料。一份很好的（而且免费的）参考资料是 Allen Downey 讲并发和信号量编程的书 [D08]。这本书里有很多谜题，做一做可以加深你对信号量以及整个并发的理解。要成为真正的并发专家需要多年的努力；想掌握这个主题，关键无疑是要超越本课程所学的内容。

参考文献

[B04] "Implementing Condition Variables with Semaphores" by Andrew Birrell. December 2004. *An interesting read on how difficult implementing CVs on top of semaphores really is, and the mistakes the author and co-workers made along the way. Particularly relevant because the group had done a ton of concurrent programming; Birrell, for example, is known for (among other things) writing various thread-programming guides.*

[CB08] "Real-world Concurrency" by Bryan Cantrill, Jeff Bonwick. ACM Queue. Volume 6, No. 5. September 2008. *A nice article by some kernel hackers from a company formerly known as Sun on the real problems faced in concurrent code.*

[CHP71] "Concurrent Control with Readers and Writers" by P.J. Courtois, F. Heymans, D.L. Parnas. Communications of the ACM, 14:10, October 1971. *The introduction of the reader-writer problem, and a simple solution. Later work introduced more complex solutions, skipped here because, well, they are pretty complex.*

[D59] "A Note on Two Problems in Connexion with Graphs" by E. W. Dijkstra. Numerische Mathematik 1, 269–271, 1959. Available: http://www-m3.ma.tum.de/twiki/pub/MN0506/WebHome/dijkstra.pdf. *Can you believe people worked on algorithms in 1959? We can't. Even before computers were any fun to use, these people had a sense that they would transform the world…*

[D68a] "Go-to Statement Considered Harmful" by E.W. Dijkstra. CACM, volume 11(3), March 1968. http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD215.PDF. *Sometimes thought of as the beginning of the field of software engineering.*

[D68b] "The Structure of the THE Multiprogramming System" by E.W. Dijkstra. CACM, volume 11(5), 1968. *One of the earliest papers to point out that systems work in computer science is an engaging intellectual endeavor. Also argues strongly for modularity in the form of layered systems.*

[D72] "Information Streams Sharing a Finite Buffer" by E.W. Dijkstra. Information Processing Letters 1, 1972. http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD329.PDF. *Did Dijkstra invent everything? No, but maybe close. He certainly was the first to clearly write down what the problems were in concurrent code. However, practitioners in OS design knew of many of the problems described by Dijkstra, so perhaps giving him too much credit would be a misrepresentation.*

[D08] "The Little Book of Semaphores" by A.B. Downey. Available at the following site: http://greenteapress.com/semaphores/. *A nice (and free!) book about semaphores. Lots of fun problems to solve, if you like that sort of thing.*

[D71] "Hierarchical ordering of sequential processes" by E.W. Dijkstra. Available online here: http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD310.PDF. *Presents numerous concurrency problems, including Dining Philosophers. The wikipedia page about this problem is also useful.*

[GR92] "Transaction Processing: Concepts and Techniques" by Jim Gray, Andreas Reuter. Morgan Kaufmann, September 1992. *The exact quote that we find particularly humorous is found on page 485, at the top of Section 8.8: "The first multiprocessors, circa 1960, had test and set instructions … presumably the OS implementors worked out the appropriate algorithms, although Dijkstra is generally credited with inventing semaphores many years later." Oh, snap!*

[H87] "Aspects of Cache Memory and Instruction Buffer Performance" by Mark D. Hill. Ph.D. Dissertation, U.C. Berkeley, 1987. *Hill's dissertation work, for those obsessed with caching in early systems. A great example of a quantitative dissertation.*

[L83] "Hints for Computer Systems Design" by Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *Lampson, a famous systems researcher, loved using hints in the design of computer systems. A hint is something that is often correct but can be wrong; in this use, a `signal()` is telling a waiting thread that it changed the condition that the waiter was waiting on, but not to trust that the condition will be in the desired state when the waiting thread wakes up. In this paper about hints for designing systems, one of Lampson's general hints is that you should use hints. It is not as confusing as it sounds.*

[T99] "Re: NT kernel guy playing with Linux" by Linus Torvalds. June 27, 1999. Available: https://yarchive.net/comp/linux/semaphores.html. *A response from Linus himself about the utility of semaphores, including the throttling case we mention in the text. As always, Linus is slightly insulting but quite informative.*

作业（编程）

在这个作业中，我们要用信号量解决一些著名的并发问题。其中很多取自 Downey 那本出色的《Little Book of Semaphores》[3]，这本书很好地汇集了许多经典问题，还介绍了几个新的变体；感兴趣的读者可以去看看这本小书，找更多乐趣。

下面每道题都给出了代码骨架；你的任务是用信号量把代码补全，让它正常工作。在 Linux 上，你可以使用原生的信号量；在 Mac 上（它不支持信号量），你得先自己构建一个实现（像本章描述的那样，用锁和条件变量）。祝你好运！

问题

1. 第一个问题只是实现并测试书中描述的 **fork/join 问题**的解法。虽然书中已经讲过这个解法，但亲手把它敲一遍还是值得的；就连巴赫也会改写维瓦尔第的作品，让一位未来的大师向一位已有的大师学习。详见 `fork-join.c`。在子线程中加上 `sleep(1)` 调用，确保它确实能正常工作。
2. 现在把前面的问题稍微推广一点，研究**会合问题**（rendezvous problem）。问题是这样的：有两个线程，每个线程都即将进入代码中的会合点。在另一个线程进入这部分代码之前，哪个线程都不应该离开它。考虑用两个信号量完成这个任务，详见 `rendezvous.c`。
3. 再进一步，实现**屏障同步**（barrier synchronization）的通用解法。假设一段顺序执行的代码中有两个点，分别叫作 P1 和 P2。在 P1 和 P2 之间放一道**屏障**（barrier），就能保证所有线程都执行完 P1 之后，才会有线程执行 P2。你的任务是：编写代码，实现一个可以这样使用的 `barrier()` 函数。可以假设你知道 N（运行中的程序里线程的总数），并且所有 N 个线程都会试图进入屏障。同样，你很可能需要用两个信号量，再加上几个用来计数的整数，来实现这个方案。详见 `barrier.c`。
4. 现在来解决**读者-写者问题**（reader-writer problem），书中同样描述过它。这一次先不用担心饥饿（starvation）。详见 `reader-writer.c` 中的代码。在代码中加上 `sleep()` 调用，证明它能按你的预期工作。你能展示出饥饿问题的存在吗？
5. 我们再来看一次读者-写者问题，但这次要考虑饥饿。怎样保证所有读者和写者最终都能取得进展？详见 `reader-writer-nostarve.c`。
6. 用信号量构建一个**无饥饿互斥锁**（no-starve mutex），让任何试图获取它的线程最终都能获得它。更多信息见 `mutex-nostarve.c` 中的代码。
7. 喜欢这些问题吗？Downey 的免费书里还有很多同类问题。别忘了，玩得开心！不过，写代码的时候你总是很开心的，不是吗？

[3]地址：http://greenteapress.com/semaphores/。